Strings in Java

Senior Level: Internals, Performance and Security

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-13

Outline

1. Inside java.lang.String
2. Strings and the JVM
3. Performance
4. Security
5. Evolution of Strings in Java
6. Summary

Inside java.lang.String

What a `String` object holds (JDK 21)

codeoutput/StringFields.java — Java

```java
import java.lang.reflect.Field;
import java.lang.reflect.Modifier;

public class StringFields {
    public static void main(String[] args) {
        for (Field f : String.class.getDeclaredFields()) {
            int mod = f.getModifiers();
            if (!Modifier.isStatic(mod)) {
                System.out.println(Modifier.toString(mod) + " "
                        + f.getType().getSimpleName() + " " + f.getName());
            }
        }
    }
}
```

Found by reflection, without opening the class. Since Java 9 there is no `char[]`: the characters live in a `byte[]` plus a `coder` flag. `hash` caches `hashCode()`; `hashIsZero` (Java 13) avoids recomputing when the hash really is 0.

What a `String` object holds (JDK 21)

codeoutput/StringFields.java — Java

```java
import java.lang.reflect.Field;
import java.lang.reflect.Modifier;

public class StringFields {
    public static void main(String[] args) {
        for (Field f : String.class.getDeclaredFields()) {
            int mod = f.getModifiers();
            if (!Modifier.isStatic(mod)) {
                System.out.println(Modifier.toString(mod) + " "
                        + f.getType().getSimpleName() + " " + f.getName());
            }
        }
    }
}
```

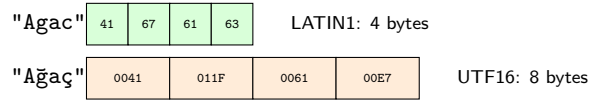
Output codeoutput/StringFields.txt

```
private final byte[] value
private final byte coder
private int hash
private boolean hashIsZero
```

Found by reflection, without opening the class. Since Java 9 there is no `char[]`: the characters live in a `byte[]` plus a `coder` flag. `hash` caches `hashCode()`; `hashIsZero` (Java 13) avoids recomputing when the hash really is 0.

Compact strings (JEP 254, Java 9)

If every character fits in one byte (Latin-1, `U+0000..U+00FF`) the string is stored with **1 byte per char** (`coder = LATIN1`); otherwise **2 bytes per char** (`coder = UTF16`).



ç (E7) alone would fit Latin-1; ğ (011F) does not, so the whole string is UTF16.

Why it pays off

Most strings in real heaps (names, keys, JSON, logs) are Latin-1. Strings are often 25%+ of the live heap, so halving their arrays saves real memory and cache bandwidth.

Cost

A branch on `coder` in every operation; mixing coders (e.g. concatenating with ğ) inflates to UTF16.

The stack trace on the freshman slides showed it: `StringLatin1.charAt`.

Measuring it

codeoutput/CompactDemo.java — Java

```java
public class CompactDemo {
    public static void main(String[] args) {
        int n = 1_000_000;
        String[] keep = new String[n];
        Runtime rt = Runtime.getRuntime();
        System.gc();
        long before = rt.totalMemory() - rt.freeMemory();
        for (int i = 0; i < n; i++) {
            keep[i] = ("student-" + (10_000_000 + i)).repeat(2); // 32 chars
        }
        System.gc();
        long after = rt.totalMemory() - rt.freeMemory();
        System.out.printf("%d x %d chars: %d MB%n",
                n, keep[0].length(), (after - before) >> 20);
    }
}
```

codeoutput/CompactStrings.sh — bash

```bash
# Run CompactDemo with and without compact strings (JEP 254).
unset JAVA_TOOL_OPTIONS                    # only if your system sets it
for flag in -XX:+CompactStrings -XX:-CompactStrings; do
    printf '%-21s ' "$flag"
    java "$flag" -cp build CompactDemo
done
```

Measuring it

codeoutput/CompactDemo.java — Java

```java
public class CompactDemo {
    public static void main(String[] args) {
        int n = 1_000_000;
        String[] keep = new String[n];
        Runtime rt = Runtime.getRuntime();
        System.gc();
        long before = rt.totalMemory() - rt.freeMemory();
        for (int i = 0; i < n; i++) {
            keep[i] = ("student-" + (10_000_000 + i)).repeat(2); // 32 chars
        }
        System.gc();
        long after = rt.totalMemory() - rt.freeMemory();
        System.out.printf("%d x %d chars: %d MB%n",
                n, keep[0].length(), (after - before) >> 20);
    }
}
```

codeoutput/CompactStrings.sh — bash

```bash
# Run CompactDemo with and without compact strings (JEP 254).
unset JAVA_TOOL_OPTIONS                  # only if your system sets it
for flag in -XX:+CompactStrings -XX:-CompactStrings; do
    printf '%-21s ' "$flag"
    java "$flag" -cp build CompactDemo
done
```

Output codeoutput/CompactStrings.txt

```
-XX:+CompactStrings   1000000 x 32 chars: 68 MB
-XX:-CompactStrings   1000000 x 32 chars: 99 MB
```

Measuring it

codeoutput/CompactDemo.java — Java

```java
public class CompactDemo {
    public static void main(String[] args) {
        int n = 1_000_000;
        String[] keep = new String[n];
        Runtime rt = Runtime.getRuntime();
        System.gc();
        long before = rt.totalMemory() - rt.freeMemory();
        for (int i = 0; i < n; i++) {
            keep[i] = ("student-" + (10_000_000 + i)).repeat(2); // 32 chars
        }
        System.gc();
        long after = rt.totalMemory() - rt.freeMemory();
        System.out.printf("%d x %d chars: %d MB%n",
                n, keep[0].length(), (after - before) >> 20);
    }
}
```

codeoutput/CompactStrings.sh — bash

```bash
# Run CompactDemo with and without compact strings (JEP 254).
unset JAVA_TOOL_OPTIONS                  # only if your system sets it
for flag in -XX:+CompactStrings -XX:-CompactStrings; do
    printf '%-21s ' "$flag"
    java "$flag" -cp build CompactDemo
done
```

Output codeoutput/CompactStrings.txt

```
-XX:+CompactStrings   1000000 x 32 chars: 68 MB
-XX:-CompactStrings   1000000 x 32 chars: 99 MB
```

One million 32-char strings: the `byte[]` needs 32 bytes instead of 64, plus a header, and each `String` object adds its own 24 bytes or so; hence about 30% less, not 50%.

Rough numbers: `totalMemory - freeMemory` after `System.gc()` is only an estimate; use JOL or a heap profiler for exact sizes.

Strings and the JVM

The string pool and deduplication

The pool (`StringTable`)

- Literals and `intern()`ed strings live in a native hash table pointing into the heap
- In the main heap since Java 7 (before: PermGen), so pooled strings are garbage collected
- Size: `-XX:StringTableSize=N`; statistics: `jcmd <pid> VM.stringtable`
- `intern()` is a hash lookup; interning everything is rarely a win

Deduplication (JEP 192)

- `-XX:+UseStringDeduplication`: the GC finds strings with equal contents and lets them **share one** `byte[]`
- The `String` objects stay distinct, so == is unchanged
- First G1 only (Java 8u20); since Java 18 also Serial, Parallel and ZGC
- Good for heaps full of repeated values (CSV, JSON, database rows)

Windows and macOS alike: `jps` lists running Java processes, `jcmd <pid> help` lists the commands; both are in the JDK's `bin` folder (Command Prompt or Terminal).

What + compiles to (JEP 280)

codeoutput/Concat.java **Java**

```java
public class Concat {
    static String greet(String name, int n) {
        return "Hello " + name + ", you have "
                + n + " messages";
    }

    public static void main(String[] args) {
        System.out.println(greet("Ayşe", 3));
    }
}
```

`javap -c` disassembles (Windows and macOS alike); `Javap.sh` also uses `javac -XDstringConcat=inline`, the pre-Java-9 way.

What + compiles to (JEP 280)

codeoutput/Concat.java — Java

```java
public class Concat {
    static String greet(String name, int n) {
        return "Hello " + name + ", you have "
                + n + " messages";
    }

    public static void main(String[] args) {
        System.out.println(greet("Ayşe", 3));
    }
}
```

`javap -c` disassembles (Windows and macOS alike); `Javap.sh` also uses `javac -XDstringConcat=inline`, the pre-Java-9 way.

Output — codeoutput/Javap.txt

```
  static java.lang.String greet(java.lang.String, int);
  Code:
  0: aload_0
  1: iload_1
  2: invokedynamic #7,  0  // InvokeDynamic #0:makeConcatWithConstants:(Lj
  7: areturn
--- javac -XDstringConcat=inline (as before Java 9) ---
  static java.lang.String greet(java.lang.String, int);
  Code:
  0: new  #7  // class java/lang/StringBuilder
  3: dup
  4: invokespecial #9  // Method java/lang/StringBuilder."<init>":()V
  7: ldc  #10  // String Hello
  9: invokevirtual #12  // Method java/lang/StringBuilder.append:(Ljava/la
  12: aload_0
  13: invokevirtual #12  // Method java/lang/StringBuilder.append:(Ljava/l
  16: ldc  #16  // String , you have
  18: invokevirtual #12  // Method java/lang/StringBuilder.append:(Ljava/l
  21: iload_1
  22: invokevirtual #18  // Method java/lang/StringBuilder.append:(I)Ljava
  25: ldc  #21  // String  messages
  27: invokevirtual #12  // Method java/lang/StringBuilder.append:(Ljava/l
  30: invokevirtual #23  // Method java/lang/StringBuilder.toString:()Ljav
  33: areturn
```

What + compiles to (JEP 280)

codeoutput/Concat.java — Java

```java
public class Concat {
    static String greet(String name, int n) {
        return "Hello " + name + ", you have "
                + n + " messages";
    }

    public static void main(String[] args) {
        System.out.println(greet("Ayşe", 3));
    }
}
```

`javap -c` disassembles (Windows and macOS alike); `Javap.sh` also uses `javac -XDstringConcat=inline`, the pre-Java-9 way.

Java 9+: one `invokedynamic` to `makeConcatWithConstants`; the JVM chooses the strategy at run time. Old class files keep the `StringBuilder` chain: recompiling is a free speed-up.

Output — codeoutput/Javap.txt

```
  static java.lang.String greet(java.lang.String, int);
  Code:
  0: aload_0
  1: iload_1
  2: invokedynamic #7,  0  // InvokeDynamic #0:makeConcatWithConstants:(Lj
  7: areturn
--- javac -XDstringConcat=inline (as before Java 9) ---
  static java.lang.String greet(java.lang.String, int);
  Code:
  0: new  #7  // class java/lang/StringBuilder
  3: dup
  4: invokespecial #9  // Method java/lang/StringBuilder."<init>":()V
  7: ldc  #10  // String Hello
  9: invokevirtual #12  // Method java/lang/StringBuilder.append:(Ljava/la
  12: aload_0
  13: invokevirtual #12  // Method java/lang/StringBuilder.append:(Ljava/l
  16: ldc  #16  // String , you have
  18: invokevirtual #12  // Method java/lang/StringBuilder.append:(Ljava/l
  21: iload_1
  22: invokevirtual #18  // Method java/lang/StringBuilder.append:(I)Ljava
  25: ldc  #21  // String  messages
  27: invokevirtual #12  // Method java/lang/StringBuilder.append:(Ljava/l
  30: invokevirtual #23  // Method java/lang/StringBuilder.toString:()Ljav
  33: areturn
```

How `switch` on a string works

```
switch (cmd) {
    case "start" -> run();
    case "stop"  -> halt();
    default      -> help();
}
```

Since Java 7. `javac` turns it into two switches.

```
int k = -1;
switch (cmd.hashCode()) {           // 1: on the hash
    case 109757538:                 // "start".hashCode()
        if (cmd.equals("start")) k = 0; break;
    case 3540994:                   // "stop".hashCode()
        if (cmd.equals("stop"))  k = 1; break;
}
switch (k) {                        // 2: on the index
    case 0 -> run();
    case 1 -> halt();
    default -> help();
}
```

Consequences: a `null` `cmd` throws `NullPointerException`; colliding hashes still work thanks to `equals`; cost is one hash (cached) plus one `equals`.

Performance

Benchmarking strings correctly

```
@State(Scope.Benchmark)
public class ConcatBench {
    @Param({"10", "1000"}) int n;

    @Benchmark
    public String plus() {
        String s = "";
        for (int i = 0; i < n; i++) s += i;
        return s;               // keep the result alive
    }

    @Benchmark
    public String builder() {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) sb.append(i);
        return sb.toString();
    }
}
```

Hand-made timing (as in the sophomore slides) is fooled by

- JIT warm-up and on-stack replacement
- dead-code elimination of unused results
- constant folding, GC pauses, CPU frequency

Use **JMH** (Java Microbenchmark Harness):

```
mvn archetype:generate \
  -DarchetypeGroupId=org.openjdk.jmh \
  -DarchetypeArtifactId=jmh-java-benchmark-archetype
mvn package
java -jar target/benchmarks.jar
```

Maven: macOS `brew install maven`; Windows: unzip from `maven.apache.org` and add `bin` to `PATH`. In Command Prompt write each command on one line (no \).

Performance guidelines

Do	Why
`+` for a few pieces in one expression	compiled to one optimized `invokedynamic`
`StringBuilder` (presized) in loops	avoids $O(n^2)$ copying and regrowth
`String.join`, `Collectors.joining`	clear and already efficient
`Pattern.compile` once, reuse	`s.matches(re)` recompiles every call
`split` with a single plain char	fast path without the regex engine
`toLowerCase(Locale.ROOT)` for keys	correct and locale-independent
`equals` before `intern`	interning costs a global table lookup
`-XX:+UseStringDeduplication` for repetitive heaps	measured memory win, no code change

First measure, then optimize: most string code is not the bottleneck.

The modern `String` API (Java 11–21)

codeoutput/ModernApi.java — Java

```java
import java.util.Arrays;

public class ModernApi {
    public static void main(String[] args) {
        String s = "  Java  ";
        System.out.println("[" + s.strip() + "] [" + s.stripTrailing() + "]");      // 11
        System.out.println("   ".isBlank() + " " + "ab".repeat(3));                  // 11
        System.out.println("a\nb\nc".lines().count());                 // 11
        System.out.print("x\ny\n".indent(2));                                      // 12
        String loud = "java".transform(t -> t.toUpperCase() + "!");                // 12
        System.out.println(loud);
        System.out.println("%d%% done".formatted(42));                            // 15
        String[] parts = "a1b22c".splitWithDelimiters("\\d+", 0);                  // 21
        System.out.println(Arrays.toString(parts));
        System.out.println(new StringBuilder().repeat("ab", 3).repeat('-', 2));    // 21
        System.out.println("hello".indexOf('l', 0, 3));                           // 21
    }
}
```

The comment on each line is the Java version that added the method.

The modern `String` API (Java 11–21)

codeoutput/ModernApi.java — Java

```java
import java.util.Arrays;

public class ModernApi {
    public static void main(String[] args) {
        String s = "  Java  ";
        System.out.println("[" + s.strip() + "] [" + s.stripTrailing() + "]");      // 11
        System.out.println("   ".isBlank() + " " + "ab".repeat(3));                  // 11
        System.out.println("a\nb\nc".lines().count());                 // 11
        System.out.print("x\ny\n".indent(2));                                      // 12
        String loud = "java".transform(t -> t.toUpperCase() + "!");                // 12
        System.out.println(loud);
        System.out.println("%d%% done".formatted(42));                             // 15
        String[] parts = "a1b22c".splitWithDelimiters("\\d+", 0);                  // 21
        System.out.println(Arrays.toString(parts));
        System.out.println(new StringBuilder().repeat("ab", 3).repeat('-', 2));    // 21
        System.out.println("hello".indexOf('l', 0, 3));                            // 21
    }
}
```

Output
codeoutput/ModernApi.txt

```
[Java] [  Java]
true ababab
3
  x
  y
JAVA!
42% done
[a, 1, b, 22, c]
ababab--
2
```

The comment on each line is the Java version that added the method.

Security

Secrets do not belong in a `String`

```
char[] pw = System.console()
        .readPassword("Password: ");
try {
    authenticate(pw);
} finally {
    Arrays.fill(pw, ' ');  // wipe
}
```

`Console.readPassword` does not echo and returns `char[]`; `PBEKeySpec` and `KeyStore` also take `char[]`.

A password in a `String`

- cannot be wiped (immutable)
- stays in the heap until GC, maybe longer; may be pooled or deduplicated
- shows up in heap dumps, core files and `toString()` logs

Honest limits

Copies made by the GC or by libraries can survive the wipe. It reduces the exposure window; it is not a guarantee.

Building queries from strings

codeoutput/Injection.java — Java

```java
public class Injection {
    public static void main(String[] args) {
        String user = "x' OR '1'='1";                    // typed by an attacker
        String sql = "SELECT * FROM users WHERE name = '" + user + "'";
        System.out.println(sql);                         // always true!
        String safe = "SELECT * FROM users WHERE name = ?";
        System.out.println(safe + "   with parameter: " + user);
    }
}
```

Concatenated input becomes **code**: the condition is always true. With `PreparedStatement` (`ps.setString(1, user)`) the input stays **data**. The same holds for shell commands, HTML (XSS), file paths (`../`) and log lines (newlines): never splice untrusted text into a language.

Building queries from strings

codeoutput/Injection.java — Java

```java
public class Injection {
    public static void main(String[] args) {
        String user = "x' OR '1'='1";                    // typed by an attacker
        String sql = "SELECT * FROM users WHERE name = '" + user + "'";
        System.out.println(sql);                         // always true!
        String safe = "SELECT * FROM users WHERE name = ?";
        System.out.println(safe + "   with parameter: " + user);
    }
}
```

Output — codeoutput/Injection.txt

```
SELECT * FROM users WHERE name = 'x' OR '1'='1'
SELECT * FROM users WHERE name = ?   with parameter: x' OR '1'='1
```

Concatenated input becomes **code**: the condition is always true. With `PreparedStatement` (`ps.setString(1, user)`) the input stays **data**. The same holds for shell commands, HTML (XSS), file paths (`../`) and log lines (newlines): never splice untrusted text into a language.

The Turkish i as a security bug

codeoutput/LocaleBug.java — Java

```java
import java.util.Locale;

public class LocaleBug {
    static boolean isEditCommand(String cmd) {
        return cmd.toUpperCase().equals("EDIT");       // default locale!
    }

    public static void main(String[] args) {
        Locale.setDefault(Locale.forLanguageTag("tr")); // a Turkish server
        System.out.println("edit  -> " + "edit".toUpperCase());
        System.out.println("check: " + isEditCommand("edit"));
        System.out.println("FILE  -> " + "FILE".toLowerCase());
        System.out.println("ROOT  -> " + "edit".toUpperCase(Locale.ROOT)
                + " " + "FILE".toLowerCase(Locale.ROOT));
    }
}
```

With a Turkish default locale the check silently fails and `"FILE".toLowerCase()` is `"file"` (keywords, file extensions, HTTP headers). Always pass `Locale.ROOT` for machine-readable text.

The Turkish `i` as a security bug

codeoutput/LocaleBug.java — Java

```java
import java.util.Locale;

public class LocaleBug {
    static boolean isEditCommand(String cmd) {
        return cmd.toUpperCase().equals("EDIT");       // default locale!
    }

    public static void main(String[] args) {
        Locale.setDefault(Locale.forLanguageTag("tr")); // a Turkish server
        System.out.println("edit  -> " + "edit".toUpperCase());
        System.out.println("check: " + isEditCommand("edit"));
        System.out.println("FILE  -> " + "FILE".toLowerCase());
        System.out.println("ROOT  -> " + "edit".toUpperCase(Locale.ROOT)
                + " " + "FILE".toLowerCase(Locale.ROOT));
    }
}
```

Output
codeoutput/LocaleBug.txt

```
edit  -> EDİT
check: false
FILE  -> fıle
ROOT  -> EDIT file
```

With a Turkish default locale the check silently fails and `"FILE".toLowerCase()` is `"fıle"` (keywords, file extensions, HTTP headers). Always pass `Locale.ROOT` for machine-readable text.

Catastrophic backtracking (ReDoS)

codeoutput/ReDoS.java — Java

```java
import java.util.regex.Pattern;

public class ReDoS {
    static double ms(Pattern p, String s) {
        long t = System.nanoTime();
        p.matcher(s).matches();                          // always false here
        return (System.nanoTime() - t) / 1e6;
    }

    public static void main(String[] args) {
        Pattern nested = Pattern.compile("(a+)+b");        // textbook case
        Pattern evil = Pattern.compile("(.*a){20}");       // still explodes
        Pattern fixed = Pattern.compile("([^a]*a){19}.*a"); // same language
        System.out.println(" n   (a+)+b  (.*a){20}  rewritten   [ms]");
        for (int n = 14; n <= 26; n += 4) {
            String s = "a".repeat(n) + "!";
            System.out.printf("%2d %8.1f %10.1f %10.2f%n",
                    n, ms(nested, s), ms(evil, s), ms(fixed, s));
        }
    }
}
```

Java 21 handles the textbook `(a+)+b`, but `(.*a){20}` grows 5 to 10× per 4 characters; the unambiguous rewrite accepts the same strings in microseconds. Limit untrusted regexes and inputs.

Catastrophic backtracking (ReDoS)

codeoutput/ReDoS.java — Java

```java
import java.util.regex.Pattern;

public class ReDoS {
    static double ms(Pattern p, String s) {
        long t = System.nanoTime();
        p.matcher(s).matches();                          // always false here
        return (System.nanoTime() - t) / 1e6;
    }

    public static void main(String[] args) {
        Pattern nested = Pattern.compile("(a+)+b");      // textbook case
        Pattern evil = Pattern.compile("(.*a){20}");     // still explodes
        Pattern fixed = Pattern.compile("([^a]*a){19}.*a"); // same language
        System.out.println(" n   (a+)+b  (.*a){20}  rewritten   [ms]");
        for (int n = 14; n <= 26; n += 4) {
            String s = "a".repeat(n) + "!";
            System.out.printf("%2d %8.1f %10.1f %10.2f%n",
                    n, ms(nested, s), ms(evil, s), ms(fixed, s));
        }
    }
}
```

Output — codeoutput/ReDoS.txt

```
 n   (a+)+b  (.*a){20}  rewritten   [ms]
14      0.2        6.7       0.05
18      0.9       36.0       0.16
22      0.1      227.2       0.11
26      0.1     2576.9       0.03
```

Java 21 handles the textbook `(a+)+b`, but `(.*a){20}` grows 5 to 10× per 4 characters; the unambiguous rewrite accepts the same strings in microseconds. Limit untrusted regexes and inputs.

Hash flooding: many keys, one hash

codeoutput/HashFlood.java — Java

```java
import java.util.*;
public class HashFlood {
    static List<String> colliding(int k) {                  // 2^k keys, one hash
        List<String> list = List.of("");
        for (int i = 0; i < k; i++) {                        // append Aa or BB
            List<String> next = new ArrayList<>();
            for (String s : list) { next.add(s + "Aa"); next.add(s + "BB"); }
            list = next;
        }
        return list;
    }
    static long fill(List<String> keys) {                   // put + get, in ms
        long t = System.nanoTime();
        Map<String, Integer> m = new HashMap<>();
        for (String s : keys) m.put(s, 1);
        for (String s : keys) m.get(s);
        return (System.nanoTime() - t) / 1_000_000;
    }
    public static void main(String[] args) {
        List<String> bad = colliding(16), good = new ArrayList<>();
        for (String s : bad) good.add(s.substring(0, 20) + good.size());
        System.out.println(bad.size() + " keys, distinct hashes: "
                + bad.stream().map(String::hashCode).distinct().count());
        fill(bad); fill(good);                                // warm up
        System.out.println("colliding " + fill(bad) + " ms, distinct " + fill(good) + " ms");
    }
}
```

Java 8+ (JEP 180): a crowded bucket becomes a tree ordered by `compareTo`, $O(\log n)$ per operation.

Hash flooding: many keys, one hash

codeoutput/HashFlood.java — Java

```java
import java.util.*;
public class HashFlood {
    static List<String> colliding(int k) {                  // 2^k keys, one hash
        List<String> list = List.of("");
        for (int i = 0; i < k; i++) {                        // append Aa or BB
            List<String> next = new ArrayList<>();
            for (String s : list) { next.add(s + "Aa"); next.add(s + "BB"); }
            list = next;
        }
        return list;
    }
    static long fill(List<String> keys) {                   // put + get, in ms
        long t = System.nanoTime();
        Map<String, Integer> m = new HashMap<>();
        for (String s : keys) m.put(s, 1);
        for (String s : keys) m.get(s);
        return (System.nanoTime() - t) / 1_000_000;
    }
    public static void main(String[] args) {
        List<String> bad = colliding(16), good = new ArrayList<>();
        for (String s : bad) good.add(s.substring(0, 20) + good.size());
        System.out.println(bad.size() + " keys, distinct hashes: "
                + bad.stream().map(String::hashCode).distinct().count());
        fill(bad); fill(good);                                // warm up
        System.out.println("colliding " + fill(bad) + " ms, distinct " + fill(good) + " ms");
    }
}
```

Output
codeoutput/HashFlood.txt

```
65536 keys, distinct hashes: 1
colliding 110 ms, distinct 6 ms
```

Java 8+ (JEP 180): a crowded bucket becomes a tree ordered by `compareTo`, $O(\log n)$ per operation.

Evolution of Strings in Java

Timeline

Java	String-related change
1.0 (1996)	`String`, synchronized `StringBuffer`
5 (2004)	`StringBuilder` (unsynchronized), `String.format`, code-point methods
7 (2011)	`switch` on strings; pool moved to the heap; `substring` copies (7u6)
8 (2014)	`String.join`, `chars()`; `HashMap` tree bins (JEP 180)
9 (2017)	compact strings (JEP 254); `invokedynamic` concatenation (JEP 280)
11 (2018)	`isBlank`, `lines`, `strip`, `repeat`
15 (2020)	text blocks (JEP 378), `formatted`, `stripIndent`
18 (2022)	UTF-8 by default (JEP 400); string deduplication in all collectors
21 (2023)	`splitWithDelimiters`, `StringBuilder.repeat`; string templates as a *preview* (JEP 430)
23 (2024)	string templates **withdrawn** for redesign: a reminder that previews can disappear

Summary

Takeaways

Internals

`byte[]` + `coder`; cached hash; pool in the heap; deduplication by the GC; `+` is an `invokedynamic` call site.

Performance

Measure with JMH. `StringBuilder` in loops, compiled `Patterns`, `Locale.ROOT`, no premature `intern`.

Security

Secrets in `char[]`; parameters, not concatenation; `Locale.ROOT` in checks; bounded, unambiguous regexes; expect colliding keys.

Portability

Explicit UTF-8 everywhere; normalize before comparing; test with Turkish and non-BMP text on Windows *and* macOS.

Projects

1. With JMH, compare `+`, `StringBuilder`, `String.join` and `Collectors.joining` for 10 to 10^6 parts; explain the curves.
2. Measure the heap of a CSV loader with and without `-XX:+UseStringDeduplication` (use `jcmd <pid> GC.heap_info`).
3. Use JOL (Java Object Layout) to print the exact size of `"Agac"` and `"Ağaç"`.
4. Write a linter that flags `toUpperCase()`/`toLowerCase()` calls without a `Locale`.
5. Build a regex sandbox that rejects patterns or inputs that run longer than 50 ms (hint: a `CharSequence` wrapper whose `charAt` checks a deadline).
6. Implement a rope (a tree of string pieces) with $O(\log n)$ insert; compare with `StringBuilder.insert`.

Questions?